QUALITY ASSURANCE ENGINEERING

TEST PLAN

DEPI Graduation Project

Document	Test Plan
Testing Scope	Desktop Web UI and REST API Functional Testing
Project Window	31/10 – 05/12

Contents

1 Introduction & Project Overview **2**
1.1 QA Team . . . 2
1.2 Systems Under Test . . . 2
1.3 Project Objectives . . . 2

2 Scope of Testing **3**
2.1 In-Scope . . . 3
2.2 Desktop Web UI Functional Coverage . . . 3
2.3 REST API Functional Coverage . . . 3
2.4 Out-of-Scope . . . 3

3 Test Strategy & Methodologies **4**
3.1 Manual Functional Testing . . . 4
3.2 REST API Testing . . . 4
3.3 BDD UI Automation . . . 4
3.4 Testing Levels and Execution Focus . . . 5

4 Test Environment & Infrastructure **5**

5 Tools & Technologies Stack **6**

6 Entry and Exit Criteria **6**
6.1 Entry Criteria . . . 6
6.2 Exit Criteria . . . 6

7 Phased Test Deliverables **7**
7.1 Manual Testing Excel Workbook . . . 7
7.2 Automation Framework Deliverable . . . 7
7.3 API Testing Deliverable . . . 7

8 Resource Allocation & Team RACI Matrix **8**
8.1 Role Allocation . . . 8

9 Agile Schedule & Sprint Milestones **9**

10 Defect Management & Jira Workflow **9**
10.1 Severity Classification and Target Response . . . 9
10.2 Defect Logging . . . 9
10.3 Jira Defect Lifecycle . . . 10

11 Risk Analysis & Mitigation Plan **10**
11.1 Risk Review . . . 11

1. Introduction & Project Overview

This Test Plan defines the functional quality assurance approach for the DEPI Graduation Project.

It establishes the testing objectives, scope, strategy, environment, tools, deliverables, team responsibilities, schedule, defect management process, and risk mitigation approach for the two confirmed Systems Under Test (SUTs).

1.1 QA Team

Team Member	Role
Ahmed Hossam Eldin Ahmed	Team Leader / QC
Yossef Ayman Mohamed	QC Engineer
Gehad Khamis Mahmoud	QC Engineer
Wafik Adel Razk	QC Engineer
Gana Emad Eldin	QC Engineer

1.2 Systems Under Test

System	Target	Testing Focus
Desktop Web UI	`https://bstackdemo.com/`	Manual functional testing and Selenium automated regression validation
REST API	`https://fakestoreapi.com`	Functional endpoint validation and API contract verification

1.3 Project Objectives

- Validate the core functional behavior of the BrowserStack Demo Store across the agreed desktop browsers.
- Verify the functional behavior, request/response payloads, and data structures of the Fake Store API endpoints.
- Design and execute structured manual test cases and document defects using Excel and Jira.
- Build a maintainable Java-based BDD automation framework using Selenium WebDriver, Cucumber, TestNG, and Page Object Model (POM).
- Prepare Postman collections for API functional validation, with optional CLI execution through Newman.
- Provide a clear, evidence-based regression result and final project sign-off.

The project is limited to functional quality validation. It does not include mobile, non-functional, or direct database testing.

2. Scope of Testing

2.1 In-Scope

- Functional Desktop Web UI testing (Manual & Automated) of `https://bstackdemo.com/`.
- Functional REST API testing (Manual & Automated) of `https://fakestoreapi.com`.
- Desktop cross-browser compatibility across Google Chrome, Brave, Microsoft Edge, and Mozilla Firefox.
- Manual test case design and bug reporting documented via Excel Sheets and tracked on Jira.
- Core BDD automated regression and smoke execution via Java, TestNG, Selenium, and Cucumber.

2.2 Desktop Web UI Functional Coverage

Target Application: BrowserStack Demo Store

Core Module	Functional Coverage
User Authentication & Session Validation	Demo logins and state persistence.
Product Catalog & Search/Filtering	Vendor filters: Apple, Samsung, Google; dynamic sorting by price/relevance.
Shopping Cart Mechanics	Add/remove items, live cart badge counter, quantity adjustments, subtotal math.
Mock Checkout & Order Processing	Address entry, fake payment confirmation modal, success messages.

2.3 REST API Functional Coverage

Target API: Fake Store API

Core Service	Functional Coverage
Authentication	/auth/login – Token generation and payload validation.
Products	/products, /products/:id, /products/categories – GET, POST, PUT, DELETE operations.
Carts	/carts, /carts/user/:id – Cart persistence, date filtering, item payloads.
Users	/users – User data structures and schema validation.

2.4 Out-of-Scope

- **NO Mobile Testing:** No mobile browsers, no native mobile apps, and no mobile viewports/emulators.
- **NO Non-Functional Testing:** No performance testing, no load/stress simulation (no JMeter), and no security/penetration testing.

- **NO Direct Database Testing:** No direct SQL queries or database server validation; integrity is evaluated purely via UI & API behavior.
- **NO Real Financial Transactions:** Only mock checkout submissions and simulated success confirmations.

3. Test Strategy & Methodologies

3.1 Manual Functional Testing

- Review the confirmed requirements and identify test conditions for each in-scope UI module.
- Prepare detailed test cases with pre-conditions, steps, test data, expected results, and execution status.
- Execute test cases on the supported desktop browsers and record actual results.
- Perform exploratory testing within the confirmed functional scope and record relevant observations.
- Log defects in the Excel Bug Reports Log and track official defect status in Jira.

3.2 REST API Testing

- Build and organize Postman requests and collections for the confirmed Fake Store API services.
- Validate HTTP methods, endpoint behavior, response status, response payloads, and expected data structures.
- Validate authentication token generation and relevant request/response fields.
- Validate product, cart, and user endpoint behavior for the specified GET, POST, PUT, and DELETE operations where supported by the target API.
- Use Newman for optional command-line collection execution.

3.3 BDD UI Automation

- Implement the automation framework in Java (JDK 17 or higher) managed by Apache Maven.
- Use Selenium WebDriver for desktop browser interactions.
- Write readable Gherkin scenarios in Cucumber .feature files using Given/When/Then syntax.
- Use TestNG for execution, assertions, annotations, and test suite configuration.
- Apply Page Object Model (POM) to separate page-level interactions from test scenarios.
- Prioritize smoke and regression coverage for the core business flows.

3.4 Testing Levels and Execution Focus

Testing Level / Type	Application	Purpose
Manual Functional Testing	BrowserStack Demo Store	Verify user-facing functional behavior and document results.
API Functional Testing	Fake Store API	Verify endpoint behavior, response payloads, and data structures.
BDD Automation	BrowserStack Demo Store	Run repeatable smoke and regression scenarios using Java, Selenium, Cucumber, and TestNG.
API Collection Execution	Fake Store API	Execute Postman collections; Newman CLI execution is optional.

4. Test Environment & Infrastructure

Environment Item	Confirmed Configuration
Platform	Desktop Web only
Desktop Operating Systems	Windows 10 / Windows 11
Target Browsers	Google Chrome, Brave, Microsoft Edge, Mozilla Firefox
Web UI SUT	https://bstackdemo.com/
REST API SUT	https://fakestoreapi.com
UI Automation	Java, Selenium WebDriver, Cucumber BDD, TestNG, Maven
API Validation	Postman Desktop; Newman CLI (optional)
Test Documentation	Microsoft Excel
Defect & Sprint Tracking	Atlassian Jira
Version Control	Git & GitHub

Execution is restricted to desktop browser environments on Windows 10/11. Mobile devices, mobile browsers, mobile viewports, and emulators are excluded.

5. Tools & Technologies Stack

Tool / Technology	Project Usage
Java (JDK 17 or higher)	Programming language for the UI automation framework.
Apache Maven	Project and dependency management through pom.xml.
Selenium WebDriver	Desktop browser interaction and UI automation.
Cucumber BDD	Gherkin feature files and readable Given/When/Then scenarios.
TestNG	Test execution, assertions, annotations, and suite configuration.
Postman Desktop	REST API request building, collections, and functional validation.
Newman	Optional command-line execution of Postman collections.
Microsoft Excel	Structured manual test cases and bug reports log.
Atlassian Jira	User story tracking, sprint boards, and official defect tracking.
Git & GitHub	Version control, code collaboration, and feature branching.

6. Entry and Exit Criteria

6.1 Entry Criteria

- The approved Test Plan and agreed scope are available to the QA team.
- The target systems are accessible: BrowserStack Demo Store and Fake Store API.
- The agreed desktop operating systems and target browsers are available for execution.
- The required tools are available: Java JDK 17 or higher, Maven, Selenium WebDriver, Cucumber, TestNG, Postman, Excel, Jira, Git, and GitHub.
- Test data and demo login details required for the planned functional scenarios are available.
- Manual test cases and API requests are prepared sufficiently to begin the relevant execution phase.
- The team responsibilities and sprint deliverables are understood.

6.2 Exit Criteria

- At least 90% pass rate is achieved for critical business flows.
- Zero unresolved Blocker or Critical defects remain at final sign-off.
- All planned in-scope test cases have an execution status recorded.
- All identified defects are documented in the Excel Bug Reports Log and tracked in Jira.

- Core UI smoke and regression scenarios have been executed and results recorded.
- Planned Postman API collections have been executed and results recorded.
- Known issues, failed tests, and unresolved lower-severity defects are documented for the final review.
- The Final Regression Execution Report and Project Sign-off are completed.

The 90% pass-rate target applies to critical business flows. Final sign-off requires zero unresolved Blocker/Critical defects; it does not imply that every test case must pass.

7. Phased Test Deliverables

Sprint	Phase	Required Deliverables
Sprint 1	Planning	Approved Test Plan Document & RACI Matrix.
Sprint 2	Analysis & Design	Manual Test Cases Excel Sheet (Sheet 1) & Exploratory Testing Notes; Postman request design.
Sprint 3	Implementation	Java Cucumber/TestNG Automation Framework (GitHub) & Postman API Collections.
Sprint 4	Execution & Completion	Jira Bug Reports Log (Excel Sheet 2 & Jira Dashboard), Final Regression Execution Report, and Project Sign-off.

7.1 Manual Testing Excel Workbook

Sheet	Required Columns
Sheet 1: Detailed Test Cases	Test ID, Module, Description, Pre-conditions, Steps, Test Data, Expected Result, Status.
Sheet 2: Bug Reports Log	Bug ID, Summary, Steps to Reproduce, Severity, Status, Jira Key.

7.2 Automation Framework Deliverable

- Java automation project managed by Maven.
- Selenium WebDriver browser interaction layer.
- Cucumber Gherkin .feature files.
- TestNG test runner, assertions, annotations, and suite configuration.
- Page Object Model (POM) architectural design.
- GitHub repository for code collaboration and feature branching.

7.3 API Testing Deliverable

- Postman collections for the confirmed Fake Store API endpoints.
- Functional endpoint validation and request/response checks.

- Optional Newman CLI execution of the Postman collections.

8. Resource Allocation & Team RACI Matrix

RACI definitions:

- R = Responsible (performs the task).
- A = Accountable (owns the outcome).
- C = Consulted.
- I = Informed.

Each deliverable has one accountable owner.

Activity / Deliverable	Ahmed Hossam	Yossef Ayman	Gehad Khamis	Wafik Adel	Gana Emad
Test Plan, scope & RACI	A/R	C	C	I	I
Manual UI test case design	A	R	R	C	I
Manual UI execution & evidence	A	R	R	C	I
API request design & Postman collections	A	C	I	R	R
Java/Selenium POM framework	A	C	R	R	I
Cucumber features & TestNG execution	A	R	R	C	I
Excel bug log & Jira defect tracking	A	R	C	R	I
Regression report & final sign-off	A/R	C	C	C	I

8.1 Role Allocation

- **Ahmed Hossam Eldin Ahmed – Team Leader / QC:** Accountable for the QA plan, scope alignment, coordination, quality review, consolidated reporting, and final sign-off.
- **Yossef Ayman Mohamed – QC Engineer:** Manual UI test design and execution, Cucumber scenario contribution, and defect evidence.
- **Gehad Khamis Mahmoud – QC Engineer:** Manual UI coverage, Selenium/POM framework implementation, and automation execution.
- **Wafik Adel Razk – QC Engineer:** API test design and collections, automation framework contribution, and defect tracking support.
- **Gana Emad Eldin – QC Engineer:** API request and collection contribution, API payload/schema checks, and supporting test documentation.

9. Agile Schedule & Sprint Milestones

Project schedule: 31/10 to 05/12.

Sprint	Dates	Milestone & Activities
Sprint 1	31/10 to 11/11	Planning, Test Plan Documentation, Scope & RACI Setup.
Sprint 2	12/11 to 19/11	Analysis & Design, Authoring Manual Test Cases in Excel, Postman request design.
Sprint 3	20/11 to 27/11	Implementation, Java POM Framework Setup, Cucumber step definitions, Postman test scripts.
Sprint 4	28/11 to 05/12	Execution & Completion, Regression run, Defect logging & tracking on Jira, Final Sign-off.

Sprint 1 starts on 31/10 and Sprint 4 ends on 05/12. The sprint dates above are the schedule baseline for this Test Plan.

10. Defect Management & Jira Workflow

10.1 Severity Classification and Target Response

Severity	Description	Target Response / Handling
Blocker	Prevents testing or blocks a critical business flow; no acceptable workaround.	Immediate team notification; prioritize investigation and fix. Must be resolved before final sign-off.
Critical	Major failure in a critical function with severe impact and no practical workaround.	High-priority investigation and fix; must be resolved before final sign-off.
Major	Important functional failure with a workaround or limited impact on other flows.	Prioritize within the active sprint; retest after the fix.
Minor	Low-impact issue, cosmetic inconsistency, or limited functional inconvenience.	Record, review, and schedule based on available sprint capacity.

10.2 Defect Logging

- Record each identified defect in the Excel Bug Reports Log using: Bug ID, Summary, Steps to Reproduce, Severity, Status, and Jira Key.
- Create or update the corresponding Jira defect for official tracking.
- Include clear reproduction steps and relevant test context so the issue can be reproduced.
- Link the Jira defect to the related user story, sprint, or test activity where applicable.
- Update the Excel Status and Jira Key when the defect lifecycle changes.

10.3 Jira Defect Lifecycle

Status	Description
Open	A defect is reported and awaits review or assignment.
In Progress	The assigned owner is investigating or working on the fix.
Fixed	A fix has been provided and is ready for QA verification.
Retest	QA is validating the fix against the reproduction steps and affected behavior.
Reopened	The issue persists or reappears after the attempted fix.
Closed	QA verification confirms the defect is resolved.

Blocker and Critical defects must be resolved before final sign-off. Major and Minor defects must remain documented with their current status and Jira reference.

11. Risk Analysis & Mitigation Plan

Risk	Potential Impact	Mitigation Strategy
Target application or API becomes unavailable	Planned execution is delayed and test evidence cannot be collected.	Check SUT availability before execution; record the interruption and resume when the target is accessible.
Unclear or changing functional behavior	Test cases may not match the expected application behavior.	Review the confirmed scope and observable behavior with the team; update affected test cases and keep changes traceable.
Browser-specific UI behavior	A flow may work in one browser and fail in another.	Execute the agreed desktop smoke coverage across Chrome, Brave, Edge, and Firefox; record browser and environment details for each defect.
Automation framework or dependency issues	Automation implementation or execution may be delayed.	Use Java JDK 17 or higher and Maven; keep the framework structure consistent with Selenium, Cucumber, TestNG, and POM; use Git feature branches for controlled collaboration.
Insufficient test data or demo account access	Authentication and business-flow scenarios may not be executable.	Identify required demo data during test design and verify access before the relevant execution.
API response or schema differs from expectations	API test cases may fail or require clarification.	Validate status codes and response payloads against the observed endpoint behavior; record discrepancies as defects or clarification items.
Defect details are incomplete	Developers may be unable to reproduce or verify reported issues.	Use clear reproduction steps, test data, severity, status, and Jira Key in the Excel log and Jira issue.
Sprint workload or schedule slippage	One or more deliverables may not be completed by the planned milestone.	Track deliverables against sprint dates, distribute work using the RACI matrix, and raise blockers early.
Scope expansion into excluded testing	Team capacity is diverted from confirmed functional deliverables.	Keep Mobile, Non-Functional, direct DB, and real financial transaction testing explicitly out of scope.

11.1 Risk Review

The QA Team Leader reviews project risks during sprint coordination. Risks affecting the agreed scope, schedule, test execution, or final exit criteria are raised to the team and tracked until an action is agreed.